GPA Calculator Project

Section 1: Program Overview

For this project, I chose to make a grade calculator. The goal of the program is to help students calculate their overall GPA using grades from multiple classes. The program asks the user to enter each class name, the numeric grade, and the number of credits for the class. After all classes are entered, the program converts the numeric grades into letter grades and then calculates the GPA. This program would be helpful for college students who want an idea of their GPA before official grades are posted. It also helps avoid mistakes that can happen when calculating GPA by hand. Overall, this program is simple but useful for everyday student life.

Section 2: Technical Requirements

Dictionary: The program uses a dictionary to store class information (Final Grade, credits). The class name is the key, and the value stores the numeric grade and the number of credits for that class.

Function: The program defines a function called convert_to_letter_grade. This function takes a numeric grade as a parameter and returns the corresponding letter grade.

Loop: The program uses a loop to allow users to enter multiple classes. The loop keeps running until the user types "done".

User Input: The program asks the user to enter the class name, numeric grade, and number of credits.

Output: The program displays each class with its letter grade and shows the final GPA at the end.

Try/Except: Try/except is used to handle invalid input, such as entering letters instead of numbers or input not in the dictionary

Section 3: Detailed Pseudocode

Pseudocode: GPA Calculator System

Create a dictionary called grade_scale with letter grades as keys and quality points as values

"A" maps to 4.0

"A-" maps to 3.7

"B+" maps to 3.3

"B" maps to 3.0

“B-” maps to 2.7

“C+” maps to 2.3

"C" maps to 2.0

"D" maps to 1.0

"F" maps to 0.0

Create an empty dictionary called classes

Display "Welcome to the Grade Calculator System"

Loop

Ask the user to type a class name (or "done" to finish)

If the user typed "done", break out of the loop

Loop to get a valid numeric grade

Try

Ask the user to enter a numeric grade

Except ValueError

Display "Invalid input. Please enter a number"

Repeat grade input loop

If the grade is not between 0 and 100

Display "Grade must be between 0 and 100"

Repeat grade input loop

Otherwise

Exit grade input loop

Loop to get valid credits

Try

Ask the user to enter the number of credits

Except ValueError

Display "Invalid input. Please enter a number"

Repeat credits input loop

If credits are less than or equal to 0

Display "Credits must be greater than 0"

Repeat credits input loop

Otherwise

Exit credits input loop

Store the numeric grade and credits in classes using the class name as the key

Define a function called convert_to_letter_grade that takes a numeric grade

If the grade is 94 or higher

Return "A"

Else if the grade is between 90 and 93

Return "A-"

Else if the grade is between 87 and 89

Return "B+"

Else if the grade is between 83 and 86

Return "B"

Else if the grade is between 80 and 82

Return "B-"

Else if the grade is between 77 and 79

Return "C+"

Else if the grade is between 70 and 76

Return "C"

Else if the grade is between 60 and 69

Return "D"

Otherwise

Return "F"

Set total_quality_points to 0
Set total_credits to 0

For each class in classes

Retrieve the numeric grade and credits

Call convert_to_letter_grade with the numeric grade

Look up the quality points in grade_scale

Multiply quality points by credits

Add the result to total_quality_points

Add credits to total_credits

Display the class name and its letter grade

If total_credits is greater than 0

Divide total_quality_points by total_credits

Store the result as GPA

Display the GPA formatted to two decimal places

Display "Calculation complete"

Section 4: Test Cases

Test Case 1: Normal Case

The user enters two classes with valid grades and credits → The program correctly displays letter grades and calculates GPA → This test checks that the program works as expected.

Test Case 2: Edge Case

The user immediately types “done” without entering any classes → The program does not crash and does not show GPA. This test checks the loop stopping condition (while loop)

Test Case 3: Invalid Input

The user types letters when a number is expected → The program displays an “ValueError” message.

The user enters invalid grade and credit → Keep asking until get the valid input → This test checks the program won’t skip if receiving a invalid input

Section 5: Design Rationale

I chose the grade calculator because it is something students actually use and can relate to. I organized the program using a loop so users can enter as many classes as they need. Using a dictionary makes more sense than using separate lists because it keeps each class connected to its grade and credits. I also used a function to handle grade conversion so the code does not repeat itself. The hardest part was definitely Pseucode because it took me a lot of time and I think it was not easy to put everything together and make it clearly explained to other people. Overall, the program is simple, organized, flexible and easy to expand later if needed.